Week 3 — Data Quality Validation Report

The file demand_enriched_corrupted.parquet contains 6,330,245 rows from 2023-01-01 to 2026-02-28. Rows before 2026-01-16 are clean and the 250,853 rows from that date onward are corrupted, so I treat the pre-cutoff part as the baseline (reference) and the post-cutoff part as the incoming batch that the validator inspects. One detail shaped the design of the checks: the corrupted window has a 0% null rate on every column. The bad values are not missing; they are the right type but wrong in meaning, so a null-rate check would say the file looks fine.

Part 1 — Issues Found

Issue 1 — Illegal trip_count values

trip_count is the model target and also feeds the lag and rolling features, so a bad value pollutes both the training signal and the lag inputs. There are three forms of bad value:

Sub-case	Condition	Affected rows	Severity
1a Negative	trip_count < 0	353	high
1b Sentinel	trip_count == 99999	147	high
1c Extreme outlier	trip_count > baseline_max * 2 (sentinel excluded)	164	medium

Why these break the model: a count cannot be negative; 99999 looks like an upstream marker for bad or missing data, and if the model reads it literally it sees a fake spike of 99,999 trips; the remaining large values blow up rolling means and lag features.

Root cause guess: 1a and 1b look like the upstream system writing error codes into a numeric field instead of producing null. 1c looks more like a unit or scaling bug in part of the pipeline.

Issue 2 — Duplicate keys

Each (PULocationID, time_bucket) pair should appear once: one zone, one 15-minute bucket, one row. The window contains 10,085 duplicate rows. Duplicates change how groupby and aggregation behave without making it obvious, and they break the assumption that the panel has one row per zone per slot. The likely cause is a batch being appended twice or a join fanning out; both produce exact-key duplicates without changing column types, which is why a schema check would not catch them.

Issue 3 — Collapsed categorical feature

cbd_pricing_active should change over time (congestion pricing is on at some times and off at others). In the corrupted window it is the constant 1 for all 250,853 rows. A constant feature has no information; the model was trained on a feature that varied, so a frozen 1 shifts the input distribution and predictions get worse without any error being raised.

I might have missed this if I had checked the whole file at once. Across the full file the column still has both 0 and 1 because the baseline varies normally. The collapse only shows up inside the recent window, which is one reason the validator runs on the incoming segment rather than the whole file.

Part 2 — Validation Approach and Graceful Degradation

Validator structure

DataQualityValidator in validation/check_data_quality.py keeps thresholds as class constants and runs three checks that match the issues one-to-one: check_value_ranges (Issue 1), check_duplicates (Issue 2), check_distributions (Issue 3). validate() returns {is_valid, num_issues, issues} where each issue is {type, severity, description, count}, so the caller can filter by severity without parsing free text.

Threshold from baseline, not hardcoded. The outlier ceiling is baseline_max * 2 (= 620 on this data), so the check scales with demand instead of needing manual re-tuning. When no baseline is provided, the validator falls back to a fixed ceiling so it still runs.

Checks should not overlap. The outlier test excludes the sentinel value with & (tc != SENTINEL_VALUE); otherwise the 147 sentinel rows would be reported once as sentinel and again as outlier, which would inflate the issue count and confuse the operator.

The validator is covered by 8 tests on synthetic data: clean data passes, each corruption is caught in isolation, and a combined case confirms multiple issues are reported independently.

Graceful degradation

Validation must not take down the API. The contract spans two functions. check_and_log_data_quality() in backend/data.py reads the file and splits it into baseline and incoming; validate_and_log() in validation/check_data_quality.py wraps the validator and does the logging. Together they give three layers of protection:

1. **File I/O is wrapped.** A missing file is logged at WARNING and the function returns; an unreadable file is logged at ERROR and returns. The API keeps serving from the clean Week 2 data either way, because the incoming batch is only being checked, never used by the model.
2. **The validator call is wrapped too.** If the validator itself raises, the exception is caught, logged at ERROR, and not re-raised.
3. **Everything goes through** logging**, not** print**.** Output goes to stdout so the container platform (Docker, GKE) captures it. A clean batch logs one INFO heartbeat; each detected issue logs a WARNING with type, severity, description, and count.

Part 3 — Strategy and Trade-offs

When validation runs

- **API startup**: every deploy or pod restart re-runs the check on the current incoming batch.
- **Daily cron in CI** at 00:00 UTC.

- **Push to main**: fast feedback whenever the validator or data.py changes.

Why daily, and why anchored to the data

My first instinct was that more often is safer. What changed my mind is that validation frequency should be anchored to how often the data arrives, not how often the API is called. The demand data lands as one batch per day, so checking once a day is enough: there is no new data to find between batches. Tying the frequency to request volume would scale cost with traffic while adding no detection value, since the data has not changed.

Log volume is a separate concern handled at the infrastructure layer (log rotation, Cloud Logging retention). Lowering the check frequency to write fewer logs would give up detection coverage to solve a logging problem, which is the wrong trade.

Why CI does not load the real data

The 74 MB corrupted parquet is in .gitignore and never reaches CI. The CI job runs the pytest suite on synthetic fixtures inside the test file, on purpose: CI catches regressions in the validation **logic**, which is the part that changes in a pull request. Validating the **actual incoming batch** runs inside the API at startup, where the real file is on disk. Keeping the two paths separate keeps CI fast and deterministic.